

Oliver Krauss

DAO & 3-Layer Architecture

ALD | MTD.ba VZ SS26

Hagenberg 05.05.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recap: Lecture 9 -- State Machines & Game AI

Last session: modelling behaviour with finite state machines

FSMs give NPCs memory: one state at a time, transitions triggered by events.

Recap: Lecture 9 -- State Machines & Game AI

Last session: modelling behaviour with finite state machines

FSMs give NPCs memory: one state at a time, transitions triggered by events.

State Machine Concepts:

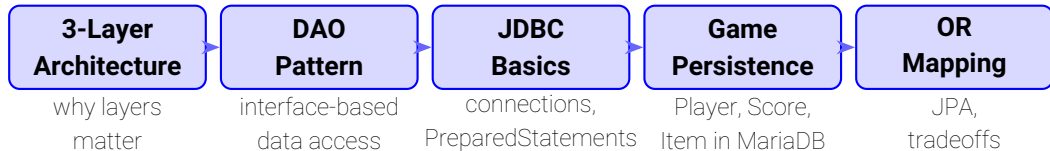
- States, transitions, initial state
- Moore vs. Mealy output models
- State diagrams as directed graphs
- Enum-based and State Pattern implementations

Game AI Application:

- NPC behaviour: patrol → chase → attack → flee
- FSM as directed graph (adjacency list)
- Strategy Pattern for transition logic
- JavaFX integration for visual feedback

Today: We fix that -- your NPC has states, but when the game closes, all progress is lost. We solve this by persisting game state in a database with clean, layered architecture.

Today's Journey



The Problem: Everything in One Place

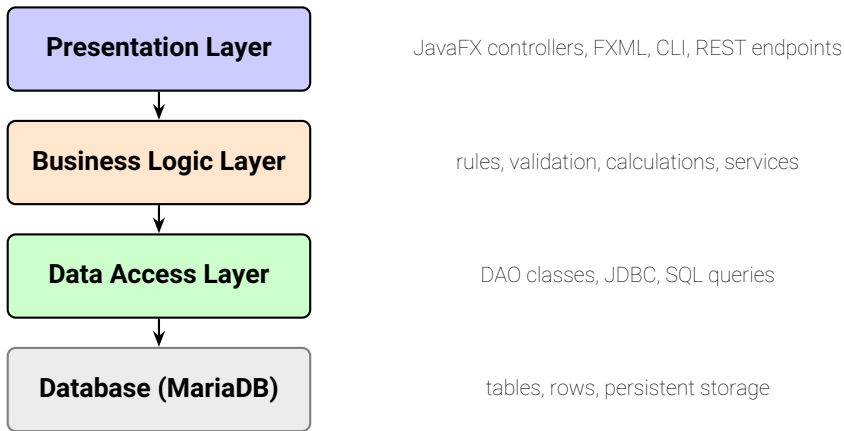
```
1  public void onSaveButtonClick() {
2      String name = nameField.getText();
3      if (name.length() < 3) {           // validation -- belongs here?
4          scoreLabel.setText("Name too short!");
5          return;
6      }
7      Connection conn = DriverManager.getConnection("jdbc:mariadb://localhost/game");
8      String sql = "INSERT INTO scores (name, points) VALUES ('"
9          + name + "', " + Integer.parseInt(pointsField.getText()) + ")";
10     Statement st = conn.createStatement();
11     st.executeUpdate(sql);
12     scoreLabel.setText("Saved!");
13 }
```

The Problem: Everything in One Place

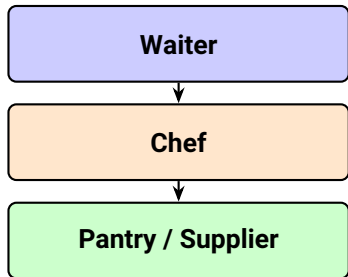
```
1  public void onSaveButtonClick() {
2      String name = nameField.getText();
3      if (name.length() < 3) {          // validation -- belongs here?
4          scoreLabel.setText("Name too short!");
5          return;
6      }
7      Connection conn = DriverManager.getConnection("jdbc:mariadb://localhost/game");
8      String sql = "INSERT INTO scores (name, points) VALUES ('"
9          + name + "', " + Integer.parseInt(pointsField.getText()) + ")";
10     Statement st = conn.createStatement();
11     st.executeUpdate(sql);
12     scoreLabel.setText("Saved!");
13 }
```

What goes wrong? To swap the database you must touch the controller. To unit-test the validation rule you need a running database. To reuse score-saving in a REST endpoint you copy and paste.

The 3-Layer Architecture



Analogy: The Restaurant



Restaurant

Our App

Waiter

Presentation Layer

Chef

Business Logic Layer

Pantry/Supplier

Data Access Layer

The **waiter** takes orders and serves food -- never cooks.

The **chef** applies recipes and rules -- never stocks shelves.

The **pantry** stores ingredients -- never deals with customers.

Each role has one job and talks only to its neighbour.

What Belongs in Each Layer?

Presentation Layer

✓ **Belongs here:**

- Read form input
- Update Labels /
TableViews
- Format numbers for
display

× **Does NOT belong:**

- SQL queries
- Business rules

What Belongs in Each Layer?

Presentation Layer

✓ **Belongs here:**

- Read form input
- Update Labels /
TableViews
- Format numbers for
display

× **Does NOT belong:**

- SQL queries
- Business rules

Business Logic Layer

✓ **Belongs here:**

- Validate input values
- Check / calculate game
rules
- Coordinate multiple
DAOs

× **Does NOT belong:**

- SQL queries
- JavaFX **Alert** dialogs

What Belongs in Each Layer?

Presentation Layer

✓ **Belongs here:**

- Read form input
- Update Labels / TableViews
- Format numbers for display

× **Does NOT belong:**

- SQL queries
- Business rules

Business Logic Layer

✓ **Belongs here:**

- Validate input values
- Check / calculate game rules
- Coordinate multiple DAOs

× **Does NOT belong:**

- SQL queries
- JavaFX **Alert** dialogs

Data Access Layer

✓ **Belongs here:**

- SQL
INSERT/SELECT/...
- Map ResultSet → objects
- Handle **SQLException**

× **Does NOT belong:**

- Validation logic
- UI code of any kind

Layer Communication Rules

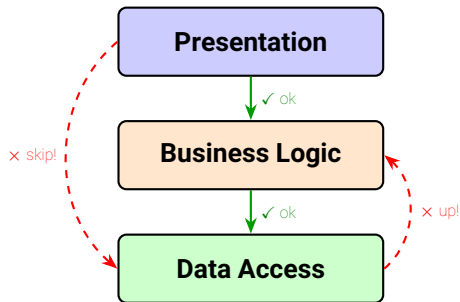
The golden rule

Each layer may only call the layer **directly below** it.

- Presentation calls Business Logic
- Business Logic calls Data Access
- Data Access calls the database

Forbidden

- Presentation jumping directly to Data Access (skipping BLL)
- Data Access calling back up into Business Logic
- Any layer calling Presentation from below



Why Bother? The Benefits

Testability

Test the Business Logic layer without a real database.

Swap the DAO with a mock that returns hardcoded data.

Replaceability

Want to switch from MariaDB to PostgreSQL or SQLite?

Only the Data Access layer changes.

Presentation and Business Logic are untouched.

Reusability

The same Business Logic can serve a JavaFX GUI **and** a REST endpoint.

No duplication of rules.

Readability & Maintenance

When a bug is reported: is it a display issue?

→ Presentation.

Wrong calculation? → Business Logic.

Bad SQL? → Data Access.

You always know where to look.

Package Structure

```
src/main/java/at/fhooe/game/  
  model/  
    Player.java  
    Score.java  
  dao/  
    Dao.java  
    PlayerDao.java  
    ScoreDao.java  
    impl/  
      MariaDbPlayerDao.java  
      MariaDbScoreDao.java  
  service/  
    PlayerService.java  
  ui/  
    GameController.java
```

Key principle

model/ - plain Java objects (POJOs), no DB, no UI logic.

dao/ - interfaces (contracts) + **impl/** (MariaDB implementations).

service/ - business logic, coordinates DAOs.

ui/ - JavaFX controllers, thin wrappers around service calls.

Convention

Never **import** `java.sql.*` outside the `dao/impl/` package.

Interactive Break #1 -- Which Layer?

Task - assign each operation to a layer

Presentation / Business Logic / Data Access

1. Format a score as "1,234 pts" for a Label
 2. Validate that a player name has 3--20 characters
 3. Execute **INSERT INTO scores ...** via JDBC
 4. Check whether a username is already taken (before creating an account)
 5. Render the top-10 leaderboard in a TableView
 6. Hash a password before storing it
- Discuss with your neighbour -- some have tricky edge cases!*

Interactive Break #1 -- Answers

#	Operation	Layer
1	Format score as "1,234 pts"	Presentation (display concern)
2	Validate name length 3--20	Business Logic (rule)
3	Execute SQL INSERT	Data Access (DAO impl)
4	Check username taken	Business Logic calls Data Access
5	Render leaderboard TableView	Presentation
6	Hash password	Business Logic (security rule)

The tricky one: #4

The *check* ("does this username exist?") is business logic. The *query* ("SELECT count(*) FROM players WHERE username = ?") is data access. Both layers cooperate: the service calls the DAO.

What Is a DAO?

Data Access Object (DAO)

A class (or interface) that **encapsulates all database operations** for a single entity type.

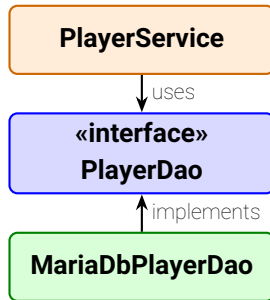
The rest of your application never writes SQL -- it calls the DAO.

Connection to Interfaces

A DAO is the **interface pattern** applied to data access.

You program against the *interface* (**PlayerDao**), not the *implementation* (**MariaDbPlayerDao**).

Swap implementations without changing a single line of business logic.



CRUD: The Four Operations

Letter	Operation	SQL	DAO method
C	Create	INSERT	<code>save(T entity)</code>
R	Read	SELECT	<code>findById(ID id), findAll()</code>
U	Update	UPDATE	<code>update(T entity)</code>
D	Delete	DELETE	<code>delete(ID id)</code>

Every persistent entity needs the same four operations

Rather than repeating them, we define a **generic** base interface once.
(This builds on the generics from Session 7.)

The Generic Dao Interface

T is a placeholder for any type, **ID** for the primary-key type.

Dao<Player, Integer> means: a DAO for **Player** entities whose primary key is an **Integer**.

```
1  public interface Dao<T, ID> {  
2  
3      Optional<T> findById(ID id);  
4  
5      List<T> findAll();  
6  
7      void save(T entity);  
8  
9      void update(T entity);  
10  
11     void delete(ID id);  
12 }
```

What **T** and **ID** mean

T - the entity type (**Player**, **Score**, **Item**).

ID - the type of the primary key (**Integer** for **int** columns).

Optional<T> means: maybe a value, maybe empty.

It avoids returning **null** from **findById(...)**.

Contract Principle

Program to the interface

*"The interface is the contract.
The implementation is the detail.
Program to the interface."*

What this means in practice

Services depend on **PlayerDao**, not on **MariaDbPlayerDao**.

So you can swap MariaDB, SQLite, or a mock without changing service code.

The Player Entity (POJO)

```
1  public class Player {  
2      private int    id;  
3      private String username;  
4      private int    level;  
5      private int    totalScore;  
6  }
```

Rule: model classes contain no database code

No `import java.sql.*` here. Pure data carrier.

The PlayerDao Interface

```
1  public interface PlayerDao extends Dao<Player, Integer> {  
2  
3      // Domain-specific query beyond basic CRUD:  
4      Optional<Player> findByUsername(String username);  
5  
6      List<Player> findTopByScore(int limit);  
7  }
```

What the service sees

Only the interface. It calls `playerDao.save(player)`,
`playerDao.findByUsername("alice")`, etc.

It has **no idea** whether the data comes from MariaDB, SQLite, or a test mock.

More DAOs: ScoreDao

```
1 public interface ScoreDao extends Dao<Score, Integer> {  
2  
3     // Domain-specific queries beyond basic CRUD:  
4     List<Score> findByPlayerId(int playerId);    // player history  
5     List<Score> findTopN(int n);                // leaderboard  
6     void deleteOlderThan(LocalDateTime cutoff); // maintenance  
7 }
```

Pattern repeats for every entity

ItemDao follows the exact same structure:

```
public interface ItemDao extends Dao<Item, Integer> { ... }
```

One interface per entity, same generic base, add domain-specific queries as needed.

Interactive Break #2 -- Design a ScoreDao

Task - sketch the interface

You are building the persistence layer for a game leaderboard.

The **Score** entity has: **id**, **playerId**, **points**, **timestamp**.

Questions to answer:

1. Which CRUD methods from **Dao<Score, Integer>** do you actually need for a leaderboard? (Think: do you ever update or delete scores in a game?)
2. What extra query methods would the leaderboard service need?
3. What should the return type of **findTopN** be -- **List<Score>** or **List<Player>**? Why?

Start with just the method signatures.

No implementation needed.

Discuss with your neighbour.

Interactive Break #2 -- Answers

```
1 public interface ScoreDao extends Dao<Score, Integer> {
2     // ALL From Dao<Score, Integer>: save(), findById(), findAll(),
3     // PLUS:
4     List<Score> findByPlayerId(int playerId);    // player's history
5     List<Score> findTopN(int n);                 // leaderboard top-N
6     void deleteOlderThan(LocalDateTime cutoff); // maintenance
7 }
```

Return type: **List<Score>** not **List<Player>**

The DAO only knows about its own entity. Joining Score with Player data is the **service's** job -- or a separate query. Keeps each DAO focused.

SQL Refresher (see RDB)

The four statements we use in this course

SQL	What it does	DAO method
INSERT INTO t (...) VALUES (...)	add a new row	save()
SELECT * FROM t WHERE ...	read rows	findById(), findAll()
UPDATE t SET col=? WHERE ...	change existing data	update()
DELETE FROM t WHERE ...	remove a row	delete()

Foreign key (FK)

A column that links a row in one table to a row in another.

`scores.player_id = 5` means “this score belongs to the player with id 5.”

The database enforces the link -- you can't create a score for a non-existent player.

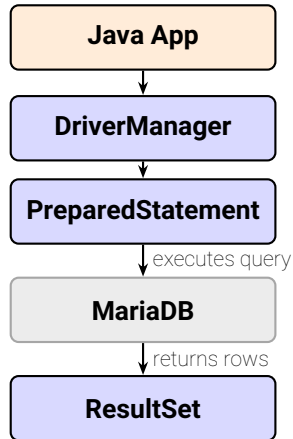
What Is JDBC?

Java Database Connectivity

Standard Java API (`java.sql.*`) for connecting to relational databases.

Works with any JDBC-compliant database through **drivers**:

- MariaDB connector
- PostgreSQL driver
- SQLite JDBC
- H2 (in-memory, great for tests)



Core JDBC Classes

Core classes

DriverManager -- obtains connections

Connection -- one open session to the DB

PreparedStatement -- parameterised SQL

ResultSet -- rows returned by a query

Using the MariaDB JDBC Connector

Maven: your app needs the MariaDB JDBC connector on the classpath.

JDBC URL format for MariaDB

`jdbc:mariadb://host:port/database`

Example: `jdbc:mariadb://localhost:3306/game_db`

Where to store credentials

Never hard-code them in source files.

Use a **db.properties** file (and add it to **.gitignore**).

Or pass as environment variables in production.

Opening a Connection

```
1  public class ConnectionProvider {  
2  
3      private static final String URL  = "jdbc:mariadb://localhost  
        :3306/game_db";  
4      private static final String USER = "gameuser";  
5      private static final String PASS = "secret";  
6  
7      public static Connection getConnection() throws SQLException {  
8          return DriverManager.getConnection(URL, USER, PASS);  
9      }  
10 }
```

Connection Lifecycle Notes

Always close your resources!

Connection, **PreparedStatement**, and **ResultSet** all hold resources.

Not closing them is a **resource leak** -- your app will eventually run out of connections.

Note: simplified for this course

This opens a **new connection per query**. In production, use a connection pool (e.g. **HikariCP**) to reuse connections efficiently.

try-with-resources: The Safe Pattern

```
1  try (Connection conn = ConnectionProvider.getConnection();
2      PreparedStatement ps = conn.prepareStatement("SELECT ...")) {
3      try (ResultSet rs = ps.executeQuery()) {
4          while (rs.next()) {
5              // process each row
6          }
7      } // rs.close() called here
8  } // ps.close() and conn.close() called here
9  catch (SQLException e) {
10     throw new RuntimeException("DB error: " + e.getMessage(), e);
11 }
```

Java automatically calls `.close()` at the end of the try block even if an exception is thrown.

Quick Check -- What Happens Without `close()`?

If you open a **Connection** and never close it, what happens eventually?

- A)** Nothing -- Java garbage-collects it automatically
- B)** The database runs out of open connections and rejects new ones
- C)** The query result gets corrupted

Quick Check -- What Happens Without `close()`?

If you open a **Connection** and never close it, what happens eventually?

- A) Nothing -- Java garbage-collects it automatically
- B) The database runs out of open connections and rejects new ones
- C) The query result gets corrupted

Answer: B

MariaDB allows at most ~150 concurrent connections by default. Each unclosed **Connection** holds one slot open. After 150 leaked connections, every new request fails with *"Too many connections"*. **try-with-resources** is your safety net.

Statement vs. PreparedStatement

Statement (AVOID AT ALL COST)

```
1  // DON'T do this:
2  String sql = "SELECT * FROM players "
3      + "WHERE username = '"
4      + username // <-- danger!
5      + "'";
6  Statement st = conn.createStatement();

7  ResultSet rs = st.executeQuery(sql);
```

Vulnerable to **SQL injection**.

```
username = "alice' OR '1'='1"
→ SELECT * FROM players WHERE
username = 'alice' OR '1'='1'
```

PreparedStatement (always use this)

```
1  // DO this instead:
2  String sql = "SELECT * FROM players
3      "
4      + "WHERE username = ?";
5  PreparedStatement ps =
6      conn.prepareStatement(sql);
7  ps.setString(1, username);
8  ResultSet rs = ps.executeQuery();
```

The ? placeholder is **escaped by the driver**.

User input can never change the query structure.

SQL Injection: Why It Matters

Classic attack

```
1  // Attacker sets username = ''; DROP TABLE players; --"
2  String sql = "SELECT * FROM players WHERE username = '" + username + "'";
3  // Executed:  SELECT * FROM players WHERE username = ''; DROP TABLE players; --'
4  //           ^--- logs in as anyone           ^--- deletes all players!
```

SQL Injection: Why It Matters

Classic attack

```
1 // Attacker sets username = ''; DROP TABLE players; --"
2 String sql = "SELECT * FROM players WHERE username = '" + username + "'";
3 // Executed:  SELECT * FROM players WHERE username = ''; DROP TABLE players; --'
4 //           ^--- logs in as anyone           ^--- deletes all players!
```

PreparedStatement blocks this entirely

```
1 PreparedStatement ps = conn.prepareStatement(
2     "SELECT * FROM players WHERE username = ?");
3 ps.setString(1, "'; DROP TABLE players; --");
4 // The driver escapes the input -- the query is treated as a literal string.
```

Rule: Every value that comes from outside your code (user input, files, APIs) must go through a `?` placeholder. No exceptions.

INSERT with PreparedStatement

```
1 public void save(Player player) {
2     String sql = "INSERT INTO players (username, level, total_score) "
3         + "VALUES (?, ?, ?)";
4
5     // RETURN_GENERATED_KEYS: ask the DB to give us back the auto-generated id
6     try (Connection conn = ConnectionProvider.getConnection();
7         PreparedStatement ps = conn.prepareStatement(
8             sql, Statement.RETURN_GENERATED_KEYS)) {
9
10        ps.setString(1, player.getUsername());
11        ps.setInt(2, player.getLevel());
12        ps.setInt(3, player.getTotalScore());
13        ps.executeUpdate();
14
15        // The DB assigned id=42 (for example) -- we read it back so our Java
16        // object has the real id instead of the placeholder 0 we passed in.
17        try (ResultSet keys = ps.getGeneratedKeys()) {
18            if (keys.next()) {
19                player.setId(keys.getInt(1));
20            }
21        }
22    } catch (SQLException e) {
23        throw new RuntimeException("save player failed", e);
24    }
25 }
```

Why generated keys?

- DB creates the real id (auto-increment).
- Java object still has a placeholder id before insert.
- **getGeneratedKeys()** syncs object and database state.

Key idea: Persist first, then store the real id in the entity.

SELECT and Mapping ResultSet

```
1 public Optional<Player> findById(Integer id) {
2     String sql = "SELECT id, username, level, total_score "
3         + "FROM players WHERE id = ?";
4     try (Connection conn = ConnectionProvider.getConnection();
5         PreparedStatement ps = conn.prepareStatement(sql)) {
6         ps.setInt(1, id);
7         try (ResultSet rs = ps.executeQuery()) {
8             if (rs.next()) {
9                 return Optional.of(mapRow(rs));
10            }
11        }
12    } catch (SQLException e) {
13        throw new RuntimeException("findById failed", e);
14    }
15    return Optional.empty();
16 }

17
18 private Player mapRow(ResultSet rs) throws SQLException {
19     return new Player(
20         rs.getInt("id"),
21         rs.getString("username"),
22         rs.getInt("level"),
23         rs.getInt("total_score")
24     );
25 }
```

Why mapRow?

- Translates DB row → domain object.
- Keeps SQL execution separate from object construction.
- Reuse mapping logic in multiple DAO methods.

Key idea: DAO returns domain objects, not raw **ResultSet**.

UPDATE with PreparedStatement

```
1  public void update(Player player) {
2      String sql = "UPDATE players SET level = ?, total_score = ? "
3                  + "WHERE id = ?";
4      try (Connection conn = ConnectionProvider.getConnection();
5           PreparedStatement ps = conn.prepareStatement(sql)) {
6          ps.setInt(1, player.getLevel()); ...
7          ps.executeUpdate();
8      } catch (SQLException e) {
9          throw new RuntimeException("update player failed", e);
10     }
11 }
```

UPDATE follows the same DAO pattern: SQL with placeholders, bind parameters, execute, wrap SQL exceptions.

DELETE with PreparedStatement

```
1  public void delete(Integer id) {
2      String sql = "DELETE FROM players WHERE id = ?";
3      try (Connection conn = ConnectionProvider.getConnection();
4           PreparedStatement ps = conn.prepareStatement(sql)) {
5          ps.setInt(1, id);
6          ps.executeUpdate();
7      } catch (SQLException e) {
8          throw new RuntimeException("delete player failed", e);
9      }
10 }
```

DELETE is the same pattern with a single id parameter. Keep mutation queries parameterized and minimal.

Transaction Basics: Auto-Commit vs Manual

The problem without transactions

Record a score (INSERT) **and** update the player total (UPDATE). App crashes in between.
Result: score recorded, **total_score** wrong -- data is inconsistent forever.

Transactions make both-or-neither happen.

What auto-commit means

- Default JDBC mode is **autoCommit = true**.
- Each statement is committed immediately.
- Good for single statements, unsafe for multi-step updates.

When to use transactions

Any time two or more changes must succeed **together**
If one fails, rollback keeps the database consistent.

Manual Transaction in JDBC

```
1  try (Connection conn = ConnectionProvider.getConnection()) {
2      conn.setAutoCommit(false); // manual mode: next statement
    starts tx
3      try {
4          scorePs.executeUpdate(); // INSERT score
5          playerPs.executeUpdate(); // UPDATE total_score
6          conn.commit();           // persist both changes
7      } catch (SQLException e) {
8          conn.rollback();         // undo both on any error
9          throw new RuntimeException("transaction failed", e);
10     } finally {
11         conn.setAutoCommit(true); // optional: restore default
    mode
12     }
13 }
```

Interactive Break #3 -- Spot the Bugs

Find at least 3 problems in this DAO method

```
1  public Optional<Player> findByUsername(String username) throws SQLException {
2      Connection conn = DriverManager.getConnection(URL, USER, PASS);
3      Statement st = conn.createStatement();
4      ResultSet rs = st.executeQuery(
5          "SELECT * FROM players WHERE username = '" + username + "'");
6      if (rs.next()) {
7          Player p = mapRow(rs);
8          // make sure score is at least 0 (business rule!)
9          if (p.getTotalScore() < 0) p.setTotalScore(0);
10         return Optional.of(p);
11     }
12     return Optional.empty();
13     // no finally, no close()
14 }
```

Interactive Break #3 -- Answers

Bug 1: SQL injection

Statement + string concatenation.

Fix: use **PreparedStatement** with ? placeholder.

Bug 3: Business logic in the DAO

```
if (p.getTotalScore() < 0)
```

p.setTotalScore(0) is a business rule.

It doesn't belong in **MariaDbPlayerDao** -- it belongs in **PlayerService**.

Fix: remove from DAO; add to service.

Bug 2: Resource leak

Connection, **Statement**, **ResultSet** are never closed.

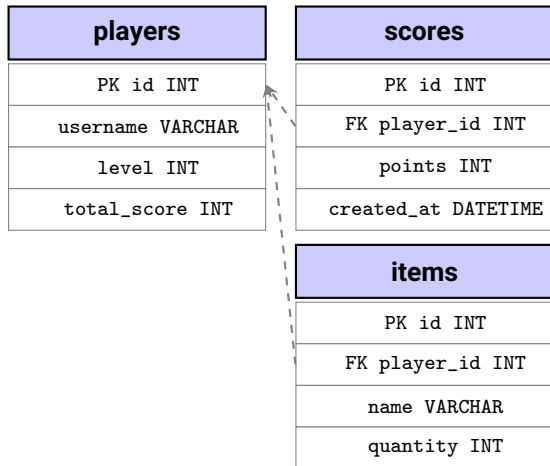
Fix: **try-with-resources** for all three.

Bonus bug (sharp eyes)

throws SQLException in a DAO method *leaks the abstraction* -- meaning an implementation detail (SQL) escapes through the interface. The controller or service now needs to handle **SQLException**, even though it's not supposed to know a database exists. Fix: catch inside the DAO and wrap in a **RuntimeException**.

Game Scenario: Player, Score, Item

Three entities to persist:



Relationships

One **Player** has many **Score** records.

One **Player** carries many **Item** records.

Each **Score** and **Item** stores a **player_id** foreign key.

MariaDB Schema

```
1 CREATE TABLE players (  
2     id            INT AUTO_INCREMENT PRIMARY KEY,  
3     username      VARCHAR(50) NOT NULL UNIQUE,  
4     level         INT          NOT NULL DEFAULT 1,  
5     total_score   INT          NOT NULL DEFAULT 0  
6 );  
7  
8 CREATE TABLE scores (  
9     id            INT AUTO_INCREMENT PRIMARY KEY,  
10    player_id     INT          NOT NULL,  
11    points        INT          NOT NULL,  
12    created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
13    FOREIGN KEY (player_id) REFERENCES players(id) ON DELETE CASCADE  
14 );  
15  
16 CREATE TABLE items (  
17     id            INT AUTO_INCREMENT PRIMARY KEY,  
18     player_id     INT          NOT NULL,  
19     name          VARCHAR(100) NOT NULL,  
20     quantity      INT          NOT NULL DEFAULT 1,  
21     FOREIGN KEY (player_id) REFERENCES players(id) ON DELETE CASCADE  
22 );
```

Key: **ON DELETE CASCADE**

When a player is deleted, all their scores and items are automatically deleted too. Without it, deleting a player would fail with a foreign key constraint error.

Layer Responsibilities

1) UI / Controller

- Handles clicks, text input, and labels
- Calls service methods
- Shows success/error feedback

2) Service (BLL)

- Validates input
- Enforces business rules
- Orchestrates one or more DAO calls

3) DAO (DAL)

- Runs SQL (**SELECT, INSERT, UPDATE**)
- Maps rows to Java objects
- Hides database details from upper layers

4) Database

- Stores durable data
- Enforces constraints and foreign keys
- Returns generated IDs and result rows

Separation Rule

Each layer has one responsibility and must **not** leak its concerns into other layers.

One Request Journey: Register Player

1. User clicks *Register* in the UI.
2. Controller calls **playerService.registerPlayer(username)**.
3. Service validates username and checks business rules.
4. Service asks DAO to persist the new player.
5. DAO executes SQL and receives generated player ID.
6. Service returns the domain object to controller.
7. Controller updates UI with success or error message.

Wiring: Where Concrete Classes Are Created

- Create concrete implementations at startup (**App** / bootstrap).
- Inject them downward (DAO → Service → Controller).
- Upper layers depend on interfaces, not concrete DB classes.

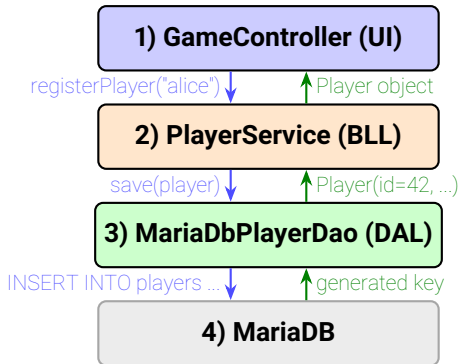
Key Idea

new MariaDbPlayerDao() should appear once in the composition root, not across the app.

Why this matters

Swapping MariaDB to another persistence technology should require minimal changes outside wiring.

The Full Stack in One Diagram



This Seems Like Lots of Repetitive Work

MariaDbPlayerDao

```
save(Player)
findById(id)
findAll()
update(Player)
delete(id)
+ mapRow(rs)
```

MariaDbScoreDao

```
save(Score)
findById(id)
findAll()
update(Score)
delete(id)
+ mapRow(rs)
```

MariaDbItemDao

```
save(Item)
findById(id)
findAll()
update(Item)
delete(id)
+ mapRow(rs)
```

For N entities: $N \times \sim 80$ lines of near-identical DAO code

Every entity gets the same **PreparedStatement** boilerplate -- just with different table names and column lists. Isn't this something a framework could handle?

What Is OR Mapping?

Object-Relational Mapping (ORM)

A framework that **generates the SQL boilerplate** for you, based on annotations on your Java classes.

You describe *how* a class maps to a table. The framework handles INSERT/SELECT/UPDATE/DELETE.

The Impedance Mismatch

OOP and relational databases think differently:

- OOP: objects, inheritance, object references
- SQL: tables, rows, foreign keys, joins

ORM bridges this gap automatically.

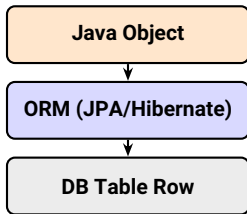
JPA and Hibernate in Practice

Java Standard: JPA

JPA (Jakarta Persistence API) -- the Java specification.

Hibernate -- the most popular implementation.

Spring Boot uses Hibernate under the hood via **Spring Data JPA**.



JPA: Annotating Your Entities

```
1  @Entity
2  @Table(name = "players")
3  public class Player {
4      @Id
5      @GeneratedValue(strategy = GenerationType.
        IDENTITY)
6      private int id;
7
8      @Column(name = "username")
9      private String username;
10     @Column(name = "level")
11     private int level;
12     @Column(name = "total_score")
13     private int totalScore;
14 }
```

Key annotations

@Entity -- this class maps to a table

@Id -- this field is the primary key

@GeneratedValue --
auto-increment

@Column -- maps field to a column

No SQL in sight

No **PreparedStatement**, no
mapRow, no **ResultSet**.

JPA Relationships: @OneToMany

```
1  @Entity
2  public class Player {
3      @OneToMany(mappedBy = "player",
4                  cascade = CascadeType.ALL, orphanRemoval
5                      = true)
6      private List<Score> scores;
7  }
```

Meaning

mappedBy = "player" -- inverse side of relation

cascade = ALL -- parent operations cascade to children

orphanRemoval = true -- remove detached child rows

Concept

One **Player** can own many **Score** entries.

JPA Relationships: @ManyToMany

```
1  @Entity
2  public class Player {
3      @ManyToMany
4      @JoinTable(
5          name = "player_items",
6          joinColumns = @JoinColumn(name = "
              player_id"),
7          inverseJoinColumns = @JoinColumn(name = "
              item_id"))
8      private Set<Item> inventory;
9  }
```

What happens in DB

JPA creates/uses a join table:

- `player_items(player_id, item_id)`
- One row = one relation link
- No duplicate links if key is constrained

Use carefully

@ManyToMany is convenient, but explicit join entities are often clearer when you need extra attributes (e.g. quantity, timestamp, role).

JPA in Action: No More Boilerplate

```
1  EntityManager em = emf.createEntityManager();
2
3  // INSERT -- id assigned by DB automatically
4  em.getTransaction().begin();
5  em.persist(player);
6  em.getTransaction().commit();
7  // SELECT by primary key
8  Player p = em.find(Player.class, 42);
9  // DELETE
10 em.getTransaction().begin();
11 em.remove(em.find(Player.class, 42));
12 em.getTransaction().commit();
13 // Custom query (class names, not tables)
14 List<Player> top = em.createQuery(
15     "SELECT p FROM Player p " +
16     "ORDER BY p.totalScore DESC",
17     Player.class)
18     .setMaxResults(10)
19     .getResultList();
```

What replaced what

JDBC	JPA
PreparedStatement	EntityManager
executeUpdate()	persist()
mapRow(rs)	(automatic)
hand-written SQL	JPQL

Note

JPQL uses **class and field names**,
not table and column names.

Player p not players p.

Tradeoffs: ORM vs. Plain JDBC

	ORM (JPA / Hibernate)	Plain JDBC
Boilerplate	Minimal	~80 lines per entity
SQL control	Limited (JPQL, Criteria API)	Full, hand-crafted
Performance	Opaque; framework decides	Predictable, tunable
Debugging	Generated SQL, hard to trace	You wrote it
DB portability	Excellent (swap dialect)	Rewrite queries
Learning curve	High (JPA specification)	Lower

Neither wins universally

Large enterprise apps with many CRUD-heavy entities lean toward ORM.

Performance-critical paths, complex reporting, or small projects lean toward JDBC.

A **hybrid** is common: ORM for standard CRUD, native SQL for hot paths.

The Cascade Dilemma

DB-level cascade (what we used)

```
1 FOREIGN KEY (player_id)
2 REFERENCES players(id)
3 ON DELETE CASCADE
```

- One SQL clause -- DB deletes all child rows in a single operation
- Works even if someone bypasses the Java app
- No Java objects loaded into memory
- Extremely fast for large collections

ORM-level cascade (what JPA does)

```
1 @OneToMany(
2     cascade = CascadeType.ALL,
3     orphanRemoval = true)
4 private List<Score> scores;
```

- JPA loads all **Score** objects into memory first
- Then issues one DELETE per score
- Java lifecycle hooks (auditing, events)
- Slow for large collections

Using both simultaneously is dangerous. Choose ONE.

The N+1 Query Problem

```
1 // Fetch all players (1 query):
2 List<Player> players = em.createQuery(
3     "SELECT p FROM Player p",
4     Player.class).getResultList();
5 // Access each player's scores:
6 for (Player p : players) {
7     // Lazy load: fires 1 extra query
8     // PER player!
9     System.out.println(p.getScores().size());
10 }
11 // Total: 1 + N queries instead of 1 JOIN
```

```
1 // Fix: JOIN FETCH loads everything at once
2 List<Player> players = em.createQuery(
3     "SELECT p FROM Player p " +
4     "JOIN FETCH p.scores",
5     Player.class).getResultList();
```

Easy to miss

Accessing a lazy-loaded collection looks like a simple getter -- but it fires a database round-trip.

With JDBC you would have written the JOIN yourself and there is no surprise.

Rule of thumb

Always check the SQL your ORM generates during development. Hibernate can log all queries:

show_sql = true

When to Choose ORM vs. JDBC

Lean toward ORM when...

- Many entities, app is CRUD-heavy
- Team already knows JPA
- Rapid prototyping or standard business logic
- Database portability is a requirement

Lean toward JDBC when...

- Performance-critical queries
- Complex reporting or aggregations
- Small project or learning context
- Fine-grained SQL control is needed

The hybrid approach (most common in practice)

ORM for standard CRUD entities, JDBC for hot paths and batch operations.

Both **`EntityManager.createNativeQuery()`** and Spring Data's **`@Query(nativeQuery=true)`** let you mix them in one project.

Interactive Break -- ORM or JDBC?

For each scenario: would you use ORM, JDBC, or both? Discuss with your neighbour.

1. Register a new player (simple INSERT, one entity).
2. Generate the top-10 leaderboard with a complex score aggregation across multiple tables.
3. Nightly maintenance job that deletes all scores older than 6 months -- potentially millions of rows.
4. Your company must support both MariaDB and PostgreSQL with a single codebase.

Interactive Break -- Answers

#	Scenario	Verdict
1	Register player (simple INSERT)	ORM -- less boilerplate, no SQL benefit
2	Leaderboard, complex aggregation	JDBC / native query -- full SQL control
3	Batch delete millions of scores	JDBC -- ORM loads all rows into memory
4	Support MariaDB & PostgreSQL	ORM -- swap dialect, no query rewrites

Scenario #3 -- the key insight

DELETE FROM scores WHERE created_at < ? is one SQL statement the DB executes instantly.

With ORM cascade, JPA fetches every matching **Score** object into memory, then deletes them one by one.

For millions of rows: seconds vs. potentially hours.

Testing Strategies for Database Applications

Pattern 1: Mocking / Fakes

- Replace DAO with mocks or fake implementations
- Focus on service logic and business rules
- Very fast and deterministic
- Great for unit tests
- Tradeoff: does not validate real SQL, schema mappings, or transactions

Pattern 2: In-Memory Test DB

- Run tests against H2/SQLite/Testcontainers
- Verify repositories, mappings, and queries
- Catches SQL/JPA issues early
- Better for integration tests
- Tradeoff: slower setup/execution and more test infrastructure

Recommended in practice

Use both: many fast unit tests with mocks/fakes, plus fewer integration tests with a real or in-memory database.

Short Recall

Without looking at your notes -- answer these:

1. Name the three layers. Which direction can calls go?
2. What is the one rule for `import java.sql.*`?
3. Name two things that must **never** appear inside a DAO class.
4. What Java construct guarantees `Connection.close()` is always called?
5. ORM check: What problem does JPA/Hibernate solve, and one common pitfall?

Short Recall

Without looking at your notes -- answer these:

1. Name the three layers. Which direction can calls go?
2. What is the one rule for `import java.sql.*`?
3. Name two things that must **never** appear inside a DAO class.
4. What Java construct guarantees `Connection.close()` is always called?
5. ORM check: What problem does JPA/Hibernate solve, and one common pitfall?

1. Presentation / Business Logic / Data Access -- downward only
2. Only inside `dao/impl/` packages
3. UI code and business rules (also: SQL injection via string concat)
4. **try-with-resources**
5. Reduces mapping boilerplate between objects and tables; pitfall: N+1 queries

Lecture Summary

Architecture

- 3 layers: UI → Service → DAO
- Each layer talks only to the one below
- Implement against interfaces!

DAO + JDBC Essentials

- One DAO per aggregate/entity (CRUD + domain queries)
- Use **PreparedStatement**; never concatenate SQL strings
- Use transactions for atomic multi-step operations

ORM (JPA/Hibernate)

- **@Entity**, **@Id**, relations replace mapping boilerplate
- Be explicit with cascades and ownership
- Hybrid is common: ORM for CRUD, SQL/JDBC for hot paths

Testing Strategy

- Unit tests: mocks
- Integration tests: in-memory/real DB